ASSESSMENT SOLUTIONS

Module 4 - SQL for Data Analytics

Assessment Code: M4-A1

PROGRAM	STACK	TOTAL TIME
Data Analytics	SQL (MySQL)	3 Hours

REFERENCE ANSWER DOCUMENT

Worked explanations and MySQL solutions for Sections A, B, C and D of the supplied assessment.

Before submission

The assessment instructions require your own schema, sample data, SQL file and screenshots of query outputs. Treat this document as a worked reference: run the scripts yourself, verify the results in your MySQL environment, and make sure you understand each query before submitting.

Document Map

Section	Contents
A - Concept Application	GROUP BY/HAVING, JOIN choice, subqueries, ranking, CASE, indexes
B - Practical Coding	Four task schemas, sample data, SELECT/JOIN/aggregation/CTE queries
C - Mini Capstone	Four-table schema, sample records, reports, window functions and VIEW
D - AI-Augmented Learning	Exact prompt, AI draft, corrected query and debugging note

General Setup and Assumptions

The supplied assessment asks for MySQL solutions, realistic sample data, and clearly labelled tasks. The practical examples below use separate databases for each task so that similarly named tables do not conflict.

- The scripts target MySQL 8.0 or later because they use CTEs and window functions.
- Run each task's CREATE DATABASE / USE statements before running its table and query code.
- Sample names, phone numbers, amounts and dates are fictional demonstration data.
- For date filters, the queries use CURDATE() so the date range is evaluated when the query is run.
- Where the assessment leaves a business rule open, the assumption is stated near the relevant query.

SECTION A - CONCEPT APPLICATION

Answer all six scenario-based questions from the assessment.

A1. GROUP BY and HAVING for Restaurant Revenue

Scenario: Report restaurants whose total revenue exceeded Rs 1,00,000 last month, but only if they received at least 50 orders.

Approach: Filter the rows to last month's date range, group the remaining orders by restaurant, calculate revenue and order count, then use HAVING to keep only groups that satisfy both aggregate conditions.

```
SELECT
    restaurant_name,
    SUM(total_amount) AS total_revenue,
    COUNT(*) AS total_orders
FROM delivery_orders
WHERE order_date >= DATE_FORMAT(CURDATE() - INTERVAL 1 MONTH, '%Y-%m-01')
  AND order_date <  DATE_FORMAT(CURDATE(), '%Y-%m-01')
GROUP BY restaurant_name
HAVING SUM(total_amount) > 100000
   AND COUNT(*) >= 50;
```

Why HAVING is appropriate:

- WHERE filters individual rows before grouping and aggregation.
- HAVING filters grouped results after SUM() and COUNT() have been calculated.
- Replacing HAVING with WHERE for SUM(total_amount) or COUNT(*) would fail because aggregate functions cannot be used in the WHERE clause at that query level.

Date-range assumption

The query treats 'last month' as the complete previous calendar month. It counts all rows in that period. If the business wants only completed orders, add an appropriate order_status condition in WHERE.

A2. Show Every Customer, Including Customers Without Orders

Scenario: Return every registered customer and their most recent order details, including customers who have never ordered.

JOIN choice: Use LEFT JOIN from customers to a ranked set of orders. LEFT JOIN keeps every customer from the left table. Customers without orders receive NULL in the order columns.

```
WITH ranked_orders AS (
    SELECT
        order_id,
        customer_id,
        restaurant_id,
```

```
        order_date,
        total_amount,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date DESC, order_id DESC
        ) AS rn
    FROM orders
)
SELECT
    c.customer_id,
    c.customer_name,
    ro.order_id,
    ro.restaurant_id,
    ro.order_date,
    ro.total_amount
FROM customers AS c
LEFT JOIN ranked_orders AS ro
    ON ro.customer_id = c.customer_id
   AND ro.rn = 1
ORDER BY c.customer_id;
```

Comparison with INNER JOIN:

- LEFT JOIN preserves all customers, whether or not they have an order.
- INNER JOIN returns only customers with at least one matching order.
- If INNER JOIN were used, customers who have never placed an order would disappear from the result.

ROW_NUMBER() assigns 1 to the most recent order for each customer. The extra order_id sort makes the choice deterministic when two orders have the same order_date.

A3. Scalar Subquery for Agents Faster Than Platform Average

Scenario: Find delivery agents whose average delivery time is lower than the overall platform average, without a temporary table or view.

Most appropriate subquery: A scalar subquery is appropriate because the overall platform average is one value. Compare each agent's grouped average with that one value in HAVING.

```
SELECT
    agent_name,
    AVG(delivery_time_mins) AS avg_delivery_time
FROM deliveries
GROUP BY agent_name
HAVING AVG(delivery_time_mins) < (
    SELECT AVG(delivery_time_mins)
    FROM deliveries
);
```

Why HAVING is used:

- The outer query calculates one average per agent after GROUP BY.
- HAVING filters those grouped averages against the scalar subquery result.

What if the subquery is placed in SELECT?

A scalar subquery in SELECT can be valid, but it would add the overall platform average as an output column for every agent. It would not by itself filter out slower agents. The filtering condition belongs in HAVING for this grouped query. A subquery returning more than one row cannot be used where a single scalar value is required.

A4. RANK() Versus DENSE_RANK()

Scenario: Rank the highest-revenue restaurants within each city when two restaurants can have the same revenue.

Restaurant	Revenue	RANK()	DENSE_RANK()
Restaurant A	120,000	1	1

Restaurant	Revenue	RANK()	DENSE_RANK()
Restaurant B	100,000	2	2
Restaurant C	100,000	2	2
Restaurant D	80,000	4	3

The first three restaurants illustrate the tie. The fourth row is included to make the difference visible: RANK() leaves a gap after a tie, while DENSE_RANK() does not.

Choice: Use DENSE_RANK() if 'top 3' means the top three distinct revenue levels in each city. It returns every restaurant tied within those three levels. Use RANK() if the report should follow competition-ranking positions, such as 1, 2, 2, 4.

```
WITH restaurant_revenue AS (
    SELECT
        city,
        restaurant_name,
        SUM(total_amount) AS total_revenue
    FROM delivery_orders
    GROUP BY city, restaurant_name
)
SELECT
    city,
    restaurant_name,
    total_revenue,
    RANK() OVER (
        PARTITION BY city
        ORDER BY total_revenue DESC
    ) AS revenue_rank,
    DENSE_RANK() OVER (
        PARTITION BY city
        ORDER BY total_revenue DESC
    ) AS dense_revenue_rank
FROM restaurant_revenue;
```

Three-row tie example

With only three rows and the tie shown above, both functions produce 1, 2, 2. A fourth row is needed to show the skipped rank: RANK() gives 4, while DENSE_RANK() gives 3.

A5. CASE WHEN for Customer Spend Segmentation

Scenario: Label customers as Platinum, Gold or Standard using lifetime spend: above Rs 5,000; Rs 2,000 to Rs 5,000; and below Rs 2,000.

```
SELECT
    c.customer_id,
    c.customer_name,
    COALESCE(SUM(o.total_amount), 0) AS lifetime_spend,
    CASE
        WHEN COALESCE(SUM(o.total_amount), 0) > 5000
            THEN 'Platinum'
        WHEN COALESCE(SUM(o.total_amount), 0) >= 2000
            THEN 'Gold'
        ELSE 'Standard'
    END AS customer_tier
FROM customers AS c
LEFT JOIN orders AS o
    ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY lifetime_spend DESC;
```

Boundary and condition order:

- Above Rs 5,000 is Platinum.
- From Rs 2,000 through Rs 5,000, inclusive, is Gold.
- Below Rs 2,000 is Standard.
- Exactly Rs 5,000 is Gold because it does not satisfy the first condition (> 5000), but it satisfies the second condition (>= 2000).
- CASE checks WHEN conditions from top to bottom and returns the result for the first matching condition.

LEFT JOIN and COALESCE() keep customers with no orders and classify their lifetime spend as zero.

A6. Database Indexes and Query Performance

An index is a data structure the database engine maintains to find rows more efficiently. In MySQL InnoDB, a primary key is stored as the clustered index, while secondary indexes store indexed key values with the primary-key value used to locate the full row.

Why an index on a JOIN column can help:

- When the query matches rows using a JOIN column, an index can let MySQL locate matching rows without scanning the entire table each time.
- It can reduce the number of rows read and lower I/O, especially when the join condition is selective and the optimizer chooses the index.
- Indexes can also help some WHERE filters and ORDER BY operations, depending on the query and index definition.

When indexes can hurt:

- Each index uses storage and memory/cache space.
- INSERT, UPDATE and DELETE operations become more expensive because relevant indexes must also be maintained.
- An index on a low-selectivity column may not help much because it matches a large portion of the table.
- Too many indexes can increase maintenance overhead and give the optimizer more choices without improving the plan.

Trade-off: Evaluate read speed against storage and write overhead. Check the actual execution plan with EXPLAIN, review the columns used in JOIN and WHERE conditions, and prefer useful composite indexes over indexing every column blindly.

SECTION B - PRACTICAL CODING TASKS

Each task below has its own database to keep table names separate. Run the setup and INSERT statements before the query section.

Task 1: Menu & Order Explorer

Create food_orders with eight sample rows spanning at least three cities.

```
CREATE DATABASE IF NOT EXISTS m4a1_task1;
USE m4a1_task1;

CREATE TABLE food_orders (
    order_id INT PRIMARY KEY,
    customer_name VARCHAR(60),
    restaurant_name VARCHAR(80),
    item_ordered VARCHAR(80),
    quantity INT,
    price_per_item DECIMAL(10,2),
    order_date DATE,
    city VARCHAR(50)
);

INSERT INTO food_orders VALUES
(1, 'Aarav', 'Spice Hub', 'Paneer Wrap', 2, 180.00, '2026-10-20', 'Mumbai'),
(2, 'Diya', 'Urban Tadka', 'Biryani', 1, 320.00, '2026-10-21', 'Mumbai'),
(3, 'Kabir', 'Coastal Bowl', 'Fish Thali', 2, 450.00, '2026-10-21', 'Mumbai'),
(4, 'Meera', 'Green Spoon', 'Salad Bowl', 1, 250.00, '2026-10-22', 'Pune'),
(5, 'Riya', 'Spice Hub', 'Masala Dosa', 3, 120.00, '2026-10-23', 'Surat'),
(6, 'Arjun', 'Urban Tadka', 'Gujarati Thali', 2, 280.00, '2026-10-24', 'Mumbai'),
(7, 'Zoya', 'Cafe Bloom', 'Cold Coffee', 2, 150.00, '2026-10-24', 'Ahmedabad'),
(8, 'Nikhil', 'Coastal Bowl', 'Pasta', 1, 380.00, '2026-10-25', 'Surat');
```

B1(a). Mumbai Orders Sorted by Price

```
SELECT *
FROM food_orders
WHERE city = 'Mumbai'
ORDER BY price_per_item DESC;
```

B1(b). Distinct Restaurant Names

```
SELECT DISTINCT restaurant_name
FROM food_orders
ORDER BY restaurant_name;
```

B1(c). Top Five Most Expensive Individual Orders

```
SELECT
    order_id,
    customer_name,
    restaurant_name,
    item_ordered,
    quantity,
    price_per_item,
    quantity * price_per_item AS order_value
FROM food_orders
ORDER BY order_value DESC
LIMIT 5;
```

Interpretation

The top-five query treats an order's value as quantity multiplied by price per item. If the instructor means highest unit price instead, sort by price_per_item DESC instead of the calculated order_value.

Task 2: Restaurant Revenue Summary

Create delivery_orders with at least ten rows and four restaurants. Calculate restaurant-level revenue, order count, average order value, and revenue category.

```
CREATE DATABASE IF NOT EXISTS m4a1_task2;
USE m4a1_task2;

CREATE TABLE delivery_orders (
    order_id INT PRIMARY KEY,
    restaurant_name VARCHAR(80),
    city VARCHAR(50),
    total_amount DECIMAL(10,2),
    order_status VARCHAR(30),
    order_date DATE
);

INSERT INTO delivery_orders VALUES
(101, 'Spice Hub', 'Mumbai', 1800.00, 'Completed', '2026-10-20'),
(102, 'Spice Hub', 'Mumbai', 2200.00, 'Completed', '2026-10-21'),
(103, 'Spice Hub', 'Mumbai', 2600.00, 'Completed', '2026-10-22'),
(104, 'Cafe Aroma', 'Surat', 4000.00, 'Completed', '2026-10-20'),
(105, 'Cafe Aroma', 'Surat', 3500.00, 'Completed', '2026-10-21'),
(106, 'Cafe Aroma', 'Surat', 2500.00, 'Completed', '2026-10-22'),
(107, 'Tandoori Tales', 'Delhi', 5000.00, 'Completed', '2026-10-20'),
(108, 'Tandoori Tales', 'Delhi', 6000.00, 'Completed', '2026-10-21'),
(109, 'Tandoori Tales', 'Delhi', 3000.00, 'Completed', '2026-10-23'),
(110, 'Green Bowl', 'Pune', 1500.00, 'Completed', '2026-10-20'),
(111, 'Green Bowl', 'Pune', 2000.00, 'Completed', '2026-10-21'),
(112, 'Green Bowl', 'Pune', 2200.00, 'Completed', '2026-10-24');
```

B2(a-c). Revenue, Count, Average, HAVING and CASE

```
SELECT
    restaurant_name,
    SUM(total_amount) AS total_revenue,
    COUNT(*) AS total_orders,
    AVG(total_amount) AS average_order_value,
    CASE
        WHEN SUM(total_amount) > 10000 THEN 'High Revenue'
        ELSE 'Moderate Revenue'
    END AS revenue_category
FROM delivery_orders
GROUP BY restaurant_name
HAVING SUM(total_amount) > 5000
ORDER BY total_revenue DESC;
```

The HAVING condition keeps restaurants with revenue above Rs 5,000. Among those qualifying restaurants, CASE labels revenue above Rs 10,000 as High Revenue and all other qualifying revenue as Moderate Revenue.

Task 3: Customer-Order-Restaurant Joined Report

Create customers, restaurants and orders. Six customers and six restaurants are included. Customers 5 and 6 intentionally have no orders.

```
CREATE DATABASE IF NOT EXISTS m4a1_task3;
USE m4a1_task3;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(60),
```

```
    city VARCHAR(50),
    phone VARCHAR(20)
);

CREATE TABLE restaurants (
    restaurant_id INT PRIMARY KEY,
    name VARCHAR(80),
    cuisine_type VARCHAR(50),
    city VARCHAR(50)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    restaurant_id INT,
    amount DECIMAL(10,2),
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

INSERT INTO customers VALUES
(1, 'Aanya', 'Mumbai', '9000000001'),
(2, 'Kabir', 'Mumbai', '9000000002'),
(3, 'Meera', 'Pune', '9000000003'),
(4, 'Rohan', 'Pune', '9000000004'),
(5, 'Zoya', 'Delhi', '9000000005'),
(6, 'Arjun', 'Delhi', '9000000006');

INSERT INTO restaurants VALUES
(101, 'Mumbai Meals', 'Indian', 'Mumbai'),
(102, 'Spice Route', 'North Indian', 'Mumbai'),
(103, 'Pune Plates', 'Maharashtrian', 'Pune'),
(104, 'Deccan Diner', 'South Indian', 'Pune'),
(105, 'Delhi Darbar', 'Mughlai', 'Delhi'),
(106, 'Capital Cafe', 'Cafe', 'Delhi');

INSERT INTO orders VALUES
(1001, 1, 101, 450.00, '2026-10-20'),
(1002, 2, 102, 650.00, '2026-10-20'),
(1003, 3, 103, 550.00, '2026-10-21'),
(1004, 4, 104, 700.00, '2026-10-21'),
(1005, 1, 102, 500.00, '2026-10-22'),
(1006, 3, 104, 600.00, '2026-10-22'),
(1007, 2, 101, 420.00, '2026-10-23'),
(1008, 1, 104, 800.00, '2026-10-24');
```

B3(a). INNER JOIN: Each Order, Customer and Restaurant

```
SELECT
    o.order_id,
    c.name AS customer_name,
    r.name AS restaurant_name,
    o.amount,
    o.order_date
FROM orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.customer_id
INNER JOIN restaurants AS r
    ON o.restaurant_id = r.restaurant_id
ORDER BY o.order_id;
```

B3(b). LEFT JOIN: All Customers, Including Those Without Orders

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    c.city AS customer_city,
    o.order_id,
    o.amount,
    o.order_date,
    r.name AS restaurant_name
FROM customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id
LEFT JOIN restaurants AS r
    ON o.restaurant_id = r.restaurant_id
ORDER BY c.customer_id, o.order_date;
```

Customers without orders remain in the result. Their order and restaurant columns are NULL.

B3(c). Orders Where Customer and Restaurant Are in the Same City

```
SELECT
    o.order_id,
    c.name AS customer_name,
    c.city AS customer_city,
    r.name AS restaurant_name,
    r.city AS restaurant_city,
    o.amount
FROM orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.customer_id
INNER JOIN restaurants AS r
    ON o.restaurant_id = r.restaurant_id
WHERE c.city = r.city
ORDER BY o.order_id;
```

The sample contains an order from a Mumbai customer to a Pune restaurant. That order is excluded by the same-city condition, showing that the filter is active.

Task 4: Delivery Agent Performance Ranking

Create a deliveries table with at least ten rows and four agents. Use a CTE to calculate agent averages, rank agents from fastest to slowest, and compare each agent with the platform average.

```
CREATE DATABASE IF NOT EXISTS m4a1_task4;
USE m4a1_task4;

CREATE TABLE deliveries (
    delivery_id INT PRIMARY KEY,
    agent_name VARCHAR(60),
    restaurant_name VARCHAR(80),
    delivery_time_mins DECIMAL(6,2),
    order_date DATE
);

INSERT INTO deliveries VALUES
(1, 'Aarav', 'Spice Hub', 30, '2026-10-20'),
(2, 'Aarav', 'Urban Tadka', 32, '2026-10-21'),
(3, 'Aarav', 'Green Bowl', 28, '2026-10-22'),
(4, 'Nisha', 'Spice Hub', 25, '2026-10-20'),
(5, 'Nisha', 'Cafe Aroma', 27, '2026-10-21'),
(6, 'Nisha', 'Green Bowl', 26, '2026-10-22'),
(7, 'Imran', 'Urban Tadka', 40, '2026-10-20'),
(8, 'Imran', 'Cafe Aroma', 38, '2026-10-21'),
(9, 'Imran', 'Spice Hub', 42, '2026-10-22'),
(10, 'Sara', 'Green Bowl', 35, '2026-10-20'),
(11, 'Sara', 'Cafe Aroma', 33, '2026-10-21'),
(12, 'Sara', 'Urban Tadka', 37, '2026-10-22');
```

B4(a-d). CTE, RANK(), Scalar Subquery and CASE

```
WITH agent_stats AS (
    SELECT
        agent_name,
        AVG(delivery_time_mins) AS avg_delivery_time
    FROM deliveries
    GROUP BY agent_name
)
SELECT
    agent_name,
    avg_delivery_time,
    RANK() OVER (
        ORDER BY avg_delivery_time ASC
    ) AS speed_rank,
    (SELECT AVG(delivery_time_mins) FROM deliveries) AS platform_avg,
    CASE
        WHEN avg_delivery_time <= (
            SELECT AVG(delivery_time_mins)
            FROM deliveries
        ) THEN 'On Track'
        ELSE 'Needs Improvement'
    END AS performance_status
FROM agent_stats
ORDER BY speed_rank, agent_name;
```

Lower average delivery time means faster performance, so the ranking uses ASC. The scalar subquery returns one overall platform average. AVG() ignores NULL delivery-time values.

SECTION C - MINI CAPSTONE PROJECT

Food Delivery Analytics - SQL Backend

This capstone uses four related tables: customers, restaurants, orders and delivery_agents. The schema includes primary keys, foreign keys and 10 or more records per table across Ahmedabad, Surat and Mumbai.

C1. Create the Capstone Database and Tables

```
CREATE DATABASE IF NOT EXISTS m4a1_capstone;
USE m4a1_capstone;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(60) NOT NULL,
    city VARCHAR(50) NOT NULL,
    phone VARCHAR(20)
);

CREATE TABLE restaurants (
    restaurant_id INT PRIMARY KEY,
    restaurant_name VARCHAR(80) NOT NULL,
    cuisine_type VARCHAR(50),
    city VARCHAR(50) NOT NULL
);

CREATE TABLE delivery_agents (
    agent_id INT PRIMARY KEY,
    agent_name VARCHAR(60) NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    restaurant_id INT NOT NULL,
    agent_id INT NOT NULL,
    order_amount DECIMAL(10,2) NOT NULL,
    delivery_time_mins INT,
    order_date DATE NOT NULL,
    order_status VARCHAR(20) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),
    FOREIGN KEY (agent_id) REFERENCES delivery_agents(agent_id)
);
```

C2. Insert Sample Customers (10 Rows)

```
INSERT INTO customers VALUES
(1, 'Aarav Shah', 'Ahmedabad', '9100000001'),
(2, 'Diya Patel', 'Ahmedabad', '9100000002'),
(3, 'Kabir Mehta', 'Ahmedabad', '9100000003'),
(4, 'Meera Desai', 'Surat', '9100000004'),
(5, 'Rohan Shah', 'Surat', '9100000005'),
(6, 'Zoya Khan', 'Surat', '9100000006'),
(7, 'Ira Kapoor', 'Mumbai', '9100000007'),
(8, 'Vivaan Rao', 'Mumbai', '9100000008'),
(9, 'Anaya Joshi', 'Mumbai', '9100000009'),
(10, 'Rehan Ali', 'Mumbai', '9100000010');
```

Customers 9 and 10 are included as examples of customers who have not placed an order in this sample dataset.

C3. Insert Sample Restaurants (10 Rows)

```
INSERT INTO restaurants VALUES
(101, 'Spice Hub', 'North Indian', 'Ahmedabad'),
(102, 'Urban Thali', 'Gujarati', 'Ahmedabad'),
(103, 'Dosa Corner', 'South Indian', 'Ahmedabad'),
(104, 'Wrap House', 'Fast Food', 'Ahmedabad'),
(105, 'Tapi Treat', 'Gujarati', 'Surat'),
(106, 'Surat Spice', 'North Indian', 'Surat'),
(107, 'Diamond Dine', 'Multi-cuisine', 'Surat'),
(108, 'Mumbai Masala', 'Indian', 'Mumbai'),
(109, 'Coastal Kitchen', 'Seafood', 'Mumbai'),
(110, 'City Bites', 'Fast Food', 'Mumbai');
```

C4. Insert Sample Delivery Agents (10 Rows)

```
INSERT INTO delivery_agents VALUES
(201, 'Aakash', 'Ahmedabad'),
(202, 'Neha', 'Ahmedabad'),
(203, 'Kabir', 'Ahmedabad'),
(204, 'Ritu', 'Ahmedabad'),
(205, 'Dev', 'Surat'),
(206, 'Sana', 'Surat'),
(207, 'Arjun', 'Surat'),
(208, 'Pooja', 'Mumbai'),
(209, 'Iqbal', 'Mumbai'),
(210, 'Meera', 'Mumbai');
```

C5. Insert Sample Orders (40 Rows)

Each restaurant receives four orders, so every restaurant can appear in the revenue report filtered to more than three orders. The order dates are generated relative to CURDATE(), keeping the sample within the recent-date range when the script is run. All sample orders are marked Completed.

```
INSERT INTO orders
    (order_id, customer_id, restaurant_id, agent_id, order_amount, delivery_time_mins, order_date,
order_status)
VALUES
(5001, 1, 101, 201, 500.00, 32, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
(5002, 2, 102, 202, 900.00, 40, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5003, 3, 103, 203, 400.00, 28, DATE_SUB(CURDATE(), INTERVAL 2 DAY), 'Completed'),
(5004, 1, 104, 204, 1000.00, 35, DATE_SUB(CURDATE(), INTERVAL 3 DAY), 'Completed'),
(5005, 2, 101, 201, 600.00, 33, DATE_SUB(CURDATE(), INTERVAL 4 DAY), 'Completed'),
(5006, 3, 102, 202, 800.00, 30, DATE_SUB(CURDATE(), INTERVAL 5 DAY), 'Completed'),
(5007, 1, 103, 203, 500.00, 25, DATE_SUB(CURDATE(), INTERVAL 6 DAY), 'Completed'),
(5008, 2, 104, 204, 900.00, 31, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
(5009, 3, 101, 201, 700.00, 29, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5010, 1, 102, 202, 500.00, 38, DATE_SUB(CURDATE(), INTERVAL 2 DAY), 'Completed'),
(5011, 2, 103, 203, 550.00, 34, DATE_SUB(CURDATE(), INTERVAL 3 DAY), 'Completed'),
(5012, 3, 104, 204, 900.00, 27, DATE_SUB(CURDATE(), INTERVAL 4 DAY), 'Completed'),
(5013, 1, 101, 201, 400.00, 30, DATE_SUB(CURDATE(), INTERVAL 5 DAY), 'Completed'),
(5014, 2, 102, 202, 750.00, 32, DATE_SUB(CURDATE(), INTERVAL 6 DAY), 'Completed'),
(5015, 3, 103, 203, 650.00, 28, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
(5016, 1, 104, 204, 850.00, 33, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5017, 4, 105, 205, 600.00, 34, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
(5018, 5, 106, 206, 1000.00, 36, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5019, 6, 107, 207, 500.00, 30, DATE_SUB(CURDATE(), INTERVAL 2 DAY), 'Completed'),
(5020, 5, 105, 205, 700.00, 32, DATE_SUB(CURDATE(), INTERVAL 3 DAY), 'Completed'),
(5021, 4, 106, 206, 1100.00, 31, DATE_SUB(CURDATE(), INTERVAL 4 DAY), 'Completed'),
(5022, 6, 107, 207, 600.00, 35, DATE_SUB(CURDATE(), INTERVAL 5 DAY), 'Completed'),
(5023, 4, 105, 205, 800.00, 29, DATE_SUB(CURDATE(), INTERVAL 6 DAY), 'Completed'),
(5024, 5, 106, 206, 900.00, 38, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
(5025, 6, 107, 207, 700.00, 33, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5026, 5, 105, 205, 900.00, 30, DATE_SUB(CURDATE(), INTERVAL 2 DAY), 'Completed'),
(5027, 6, 106, 206, 1100.00, 32, DATE_SUB(CURDATE(), INTERVAL 3 DAY), 'Completed'),
(5028, 4, 107, 207, 800.00, 28, DATE_SUB(CURDATE(), INTERVAL 4 DAY), 'Completed'),
(5029, 7, 108, 208, 1200.00, 30, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
```

```
(5030, 8, 109, 209, 800.00, 33, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5031, 7, 110, 210, 600.00, 38, DATE_SUB(CURDATE(), INTERVAL 2 DAY), 'Completed'),
(5032, 8, 108, 208, 1100.00, 29, DATE_SUB(CURDATE(), INTERVAL 3 DAY), 'Completed'),
(5033, 7, 109, 209, 900.00, 35, DATE_SUB(CURDATE(), INTERVAL 4 DAY), 'Completed'),
(5034, 8, 110, 210, 650.00, 36, DATE_SUB(CURDATE(), INTERVAL 5 DAY), 'Completed'),
(5035, 7, 108, 208, 1000.00, 31, DATE_SUB(CURDATE(), INTERVAL 6 DAY), 'Completed'),
(5036, 8, 109, 209, 1000.00, 32, DATE_SUB(CURDATE(), INTERVAL 0 DAY), 'Completed'),
(5037, 7, 110, 210, 700.00, 34, DATE_SUB(CURDATE(), INTERVAL 1 DAY), 'Completed'),
(5038, 8, 108, 208, 900.00, 28, DATE_SUB(CURDATE(), INTERVAL 2 DAY), 'Completed'),
(5039, 7, 109, 209, 1100.00, 30, DATE_SUB(CURDATE(), INTERVAL 3 DAY), 'Completed'),
(5040, 8, 110, 210, 750.00, 37, DATE_SUB(CURDATE(), INTERVAL 4 DAY), 'Completed');
```

C6. Revenue Report per Restaurant

Requirement: Use GROUP BY, HAVING, SUM, COUNT and AVG. Include restaurants with more than three orders.

```
SELECT
    r.restaurant_id,
    r.restaurant_name,
    r.city,
    COUNT(o.order_id) AS total_orders,
    SUM(o.order_amount) AS total_revenue,
    AVG(o.order_amount) AS average_order_value
FROM restaurants AS r
JOIN orders AS o
    ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.restaurant_name, r.city
HAVING COUNT(o.order_id) > 3
ORDER BY total_revenue DESC;
```

C7. Customer Segmentation by Total Spend

Requirement: Platinum above Rs 5,000; Gold from Rs 2,000 to Rs 5,000; Standard below Rs 2,000. Include customers with no orders as Standard.

```
SELECT
    c.customer_id,
    c.customer_name,
    COALESCE(SUM(o.order_amount), 0) AS total_spend,
    CASE
        WHEN COALESCE(SUM(o.order_amount), 0) > 5000
            THEN 'Platinum'
        WHEN COALESCE(SUM(o.order_amount), 0) >= 2000
            THEN 'Gold'
        ELSE 'Standard'
    END AS customer_segment
FROM customers AS c
LEFT JOIN orders AS o
    ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spend DESC;
```

Exactly Rs 5,000 belongs to Gold because Platinum is strictly above Rs 5,000. Customers with no orders have total_spend 0 due to COALESCE() and are Standard.

C8. Top Three Restaurants in Each City Using DENSE_RANK()

```
WITH restaurant_revenue AS (
    SELECT
        r.restaurant_id,
        r.restaurant_name,
        r.city,
        SUM(o.order_amount) AS total_revenue
    FROM restaurants AS r
    JOIN orders AS o
```

```
        ON o.restaurant_id = r.restaurant_id
    GROUP BY r.restaurant_id, r.restaurant_name, r.city
), ranked_restaurants AS (
    SELECT
        restaurant_id,
        restaurant_name,
        city,
        total_revenue,
        DENSE_RANK() OVER (
            PARTITION BY city
            ORDER BY total_revenue DESC
        ) AS revenue_rank
    FROM restaurant_revenue
)
SELECT *
FROM ranked_restaurants
WHERE revenue_rank <= 3
ORDER BY city, revenue_rank, restaurant_name;
```

PARTITION BY city restarts the ranking for each city. DENSE_RANK() gives tied revenue values the same rank without leaving gaps.

C9. Most Recent Order for Each Customer Using ROW_NUMBER()

```
WITH ranked_orders AS (
    SELECT
        order_id,
        customer_id,
        order_date,
        order_amount,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date DESC, order_id DESC
        ) AS rn
    FROM orders
)
SELECT
    c.customer_id,
    c.customer_name,
    ro.order_id,
    ro.order_date,
    ro.order_amount
FROM customers AS c
LEFT JOIN ranked_orders AS ro
    ON ro.customer_id = c.customer_id
   AND ro.rn = 1
ORDER BY c.customer_id;
```

ROW_NUMBER() = 1 identifies one most recent order per customer. The LEFT JOIN also retains customers with no orders, showing NULL order details for them. The order_id tie-break makes the selection deterministic if dates match.

C10. Create and Query the View

Requirement: Create vw_daily_order_summary by joining all four tables and return the specified daily order fields.

```
CREATE OR REPLACE VIEW vw_daily_order_summary AS
SELECT
    o.order_date,
    c.customer_name,
    r.restaurant_name,
    da.agent_name,
    o.order_amount,
    o.delivery_time_mins
FROM orders AS o
JOIN customers AS c
    ON o.customer_id = c.customer_id
JOIN restaurants AS r
    ON o.restaurant_id = r.restaurant_id
JOIN delivery_agents AS da
    ON o.agent_id = da.agent_id;
```

Query the View for the Last Seven Calendar Days

```
SELECT *
FROM vw_daily_order_summary
WHERE order_date >= DATE_SUB(CURDATE(), INTERVAL 6 DAY)
  AND order_date <  DATE_ADD(CURDATE(), INTERVAL 1 DAY)
ORDER BY order_date DESC, customer_name;
```

Because order_date is a DATE column, this range includes today and the six preceding calendar days: seven calendar dates in total.

SECTION D - AI-AUGMENTED LEARNING

This section documents the prompt, an initial AI draft, a corrected query, and the debugging note required by the assessment.

D1. Exact Prompt Given to the AI Tool

Prompt

Write a MySQL 8 query for a food delivery platform that finds the top 3 delivery agents per city, ranked by number of completed orders. Include each agent's average delivery time alongside their rank. Only include agents with more than 5 completed orders in that city. Sort the final output by city name alphabetically, then by rank ascending. Use window functions.

D2. Example Test Table for the Query

The query below assumes an agent_delivery_log table with one row per delivery. It includes a status column so cancelled deliveries can be excluded from completed-order counts.

```
CREATE DATABASE IF NOT EXISTS m4a1_section_d;
USE m4a1_section_d;

CREATE TABLE agent_delivery_log (
    delivery_id INT AUTO_INCREMENT PRIMARY KEY,
    agent_name VARCHAR(60) NOT NULL,
    city VARCHAR(50) NOT NULL,
    delivery_time_mins DECIMAL(6,2) NULL,
    delivery_status VARCHAR(20) NOT NULL,
    order_date DATE NOT NULL
);
```

Optional compact sample-data generator: it creates 12 agents across Ahmedabad, Surat and Mumbai. Each city has agents with 8, 7, 6 and 5 completed deliveries. The query filters to more than five, leaving three qualifying agents per city.

```
INSERT INTO agent_delivery_log
    (agent_name, city, delivery_time_mins, delivery_status, order_date)
SELECT
    a.agent_name,
    a.city,
    a.base_minutes + MOD(n.n, 3) AS delivery_time_mins,
    'Completed' AS delivery_status,
    DATE_SUB(CURDATE(), INTERVAL MOD(n.n, 7) DAY) AS order_date
FROM (
    SELECT 'Asha' AS agent_name, 'Ahmedabad' AS city, 28 AS base_minutes, 8 AS completed_count
    UNION ALL SELECT 'Bharat', 'Ahmedabad', 31, 7
    UNION ALL SELECT 'Charu', 'Ahmedabad', 34, 6
    UNION ALL SELECT 'Deep', 'Ahmedabad', 36, 5
    UNION ALL SELECT 'Esha', 'Surat', 27, 8
    UNION ALL SELECT 'Farhan', 'Surat', 30, 7
    UNION ALL SELECT 'Gauri', 'Surat', 33, 6
    UNION ALL SELECT 'Harsh', 'Surat', 37, 5
    UNION ALL SELECT 'Isha', 'Mumbai', 26, 8
    UNION ALL SELECT 'Jatin', 'Mumbai', 29, 7
    UNION ALL SELECT 'Kavya', 'Mumbai', 32, 6
    UNION ALL SELECT 'Laksh', 'Mumbai', 35, 5
) AS a
JOIN (
    SELECT 1 AS n UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4
    UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8
) AS n
    ON n.n <= a.completed_count;
```

```
-- Add cancelled records to test that they are excluded from completed-order counts.
INSERT INTO agent_delivery_log
    (agent_name, city, delivery_time_mins, delivery_status, order_date)
VALUES
('Asha', 'Ahmedabad', NULL, 'Cancelled', CURDATE()),
('Farhan', 'Surat', NULL, 'Cancelled', CURDATE()),
('Jatin', 'Mumbai', NULL, 'Cancelled', CURDATE());
```

D3. AI's Initial Query (Contains Issues for Debugging)

```
WITH agent_stats AS (
    SELECT
        agent_name,
        city,
        COUNT(*) AS completed_orders,
        AVG(delivery_time_mins) AS avg_delivery_time
    FROM agent_delivery_log
    GROUP BY agent_name, city
), ranked_agents AS (
    SELECT
        agent_name,
        city,
        completed_orders,
        avg_delivery_time,
        RANK() OVER (
            PARTITION BY agent_name
            ORDER BY completed_orders DESC
        ) AS agent_rank
    FROM agent_stats
)
SELECT *
FROM ranked_agents
WHERE completed_orders > 5
  AND agent_rank <= 3
ORDER BY city, agent_rank;
```

Issues in the initial query:

- It counts every delivery row, including cancelled deliveries, because it has no delivery_status = 'Completed' filter.
- It partitions ranking by agent_name instead of city. That does not rank agents against other agents in the same city.
- It applies the more-than-five threshold after ranking instead of defining the eligible agent set in the aggregation step. Moving the condition into HAVING makes it explicit that only eligible agents enter the ranking stage.

D4. Corrected Query

```
WITH agent_stats AS (
    SELECT
        agent_name,
        city,
        COUNT(*) AS completed_orders,
        AVG(delivery_time_mins) AS avg_delivery_time
    FROM agent_delivery_log
    WHERE delivery_status = 'Completed'
    GROUP BY agent_name, city
    HAVING COUNT(*) > 5
), ranked_agents AS (
    SELECT
        agent_name,
        city,
        completed_orders,
        avg_delivery_time,
        RANK() OVER (
            PARTITION BY city
            ORDER BY completed_orders DESC
        ) AS agent_rank
    FROM agent_stats
```

```
)
SELECT
    city,
    agent_name,
    completed_orders,
    avg_delivery_time,
    agent_rank
FROM ranked_agents
WHERE agent_rank <= 3
ORDER BY city ASC, agent_rank ASC, agent_name ASC;
```

Why this version is correct:

- WHERE delivery_status = 'Completed' filters the raw rows before aggregation.
- GROUP BY agent_name, city calculates completed-order counts and average delivery time per agent per city.
- HAVING COUNT(*) > 5 filters out agents who do not meet the eligibility threshold before ranking.
- RANK() PARTITION BY city ranks agents against other agents in the same city, with the highest completed-order count ranked first.
- The final WHERE agent_rank <= 3 keeps the top three rank positions. Ties may cause more than three rows to be returned, which is normal RANK() behaviour.

NULL delivery_time_mins values are ignored by AVG() in MySQL. Such a completed delivery is still included in COUNT(*). If the business instead requires every counted delivery to have a recorded delivery time, add delivery_time_mins IS NOT NULL to the WHERE clause and state that business rule.

D5. Debugging Note (3-4 Lines)

Debugging note

I added a filter for Completed deliveries so cancelled records are not counted. I changed the ranking partition from agent_name to city so agents are ranked against others in the same city. I moved the more-than-five eligibility condition into HAVING before the ranking step. I also checked how NULL delivery times affect AVG(): MySQL ignores NULL values in the average.

Final Submission Checklist

The supplied assessment says all sections must be attempted and that coding tasks require a .sql file and screenshots of query outputs. Use this checklist before submitting.

- Section A: Write answers to all six concept questions in your own words.
- Section B: Run each task's schema, sample inserts and query scripts in MySQL.
- Section C: Run the four-table capstone schema and confirm primary-key and foreign-key constraints work.
- Section D: Keep the exact AI prompt, initial query, corrected query and 3-4 line debugging note.
- Save the final MySQL scripts in a .sql file and label each section/task clearly.
- Capture screenshots showing query text and output for the practical tasks.
- Check that dates, sample outputs and database names match your own MySQL environment before submission.

Important

This document provides a complete worked reference based on the uploaded M4-A1 assessment. Query outputs can differ if your local sample data, MySQL version, date or schema differs. Run the code and keep your own screenshots as required by the assessment.